Week 2 — AI System Development Checkpoint

Overview

During Week 2, we focused on **preparing the dataset, performing exploratory data analysis (EDA), and implementing baseline machine learning and deep learning models**.

All scripts, notebooks, and documentation are organized to allow easy execution and reproducibility of experiments. This checkpoint covers all deliverables for Week 2.

Project Structure

```
project-root/
│
├── data/
│   ├── raw/            # Raw datasets
│   └── processed/      # Train, Validation, Test CSV files
│
├── notebooks/
│   └── 01_eda.ipynb    # Exploratory Data Analysis
│
├── src/
│   ├── data_pipeline.py # Data cleaning and preprocessing
│   ├── baselines.py     # ML & DL baseline models
│   ├── cnn_model.py     # Text-CNN scaffold
│   ├── nlp_scaffold.py  # Intent classifier stub
│   └── rl_stub.py       # RL agent stub
│
└── docs/
    ├── model_card.md    # Model overview, metrics, limitations
    └── ethics_statement.md  # Ethics considerations
```

Deliverables Completed

1. Data Preparation

- Created **training, validation, and test splits**.
- Cleaned datasets and verified using Pandas (`info()`, `isnull().sum()`, sample rows).
- Ensured all CSV files are ready for model input.

2. Exploratory Data Analysis (EDA)

- Visualized **class distribution** of labels (or `inbound` messages).
- Checked for **missing values** in all datasets.
- Sampled a few messages to understand the dataset content.
- Identified basic feature relationships, such as **author ID vs inbound message**.

3. Baseline Models

- Implemented **Logistic Regression** and **Decision Tree** as ML baselines.
- Implemented **Feedforward Neural Network (PyTorch)** as a DL baseline.
- Trained models on the training set and evaluated on the validation set.
- Recorded metrics including **accuracy, precision, recall, and F1-score**.

4. CNN Experiment

- Built a **Text-CNN scaffold** for text classification.
- Ready to train with initial hyperparameters.
- Learning curves can be logged to monitor training progress.

5. NLP Component

- Created an **intent classifier stub** mapping message text to predicted intent.
- Ensures input/output matches the RL environment.

6. RL Agent Stub

- Defined the **environment**: states = messages, actions = response choices.
- Implemented a simple **reward function** (+1 for correct action, -1 for incorrect).
- Ran a few sample episodes to generate **early learning curves**.

7. Ethics & Model Card

- Drafted `model_card.md` documenting **model overview, metrics, limitations**.
- Drafted `ethics_statement.md` covering **privacy, bias, and transparency considerations**.

Key Findings

- **Class imbalance** may affect baseline model performance.
- **Small dataset** limits generalization of models.
- **Initial metrics** show ML and DL baselines working but with room for improvement.
- Early RL agent simulations confirm environment setup works.

Next Steps

1. Fine-tune **CNN model** with optimized hyperparameters.
2. Enhance **NLP intent classifier** with embeddings for better predictions.
3. Integrate **RL agent** with NLP predictions for action selection.
4. Expand **EDA** for deeper insights into the dataset.
5. Continue refining **Model Card** and **Ethics Statement**.

How to Run

1. Make sure **Python 3.11** is installed.
2. Install required packages:

pip install pandas scikit-learn torch matplotlib seaborn

3. Run baseline models:

python src/baselines.py

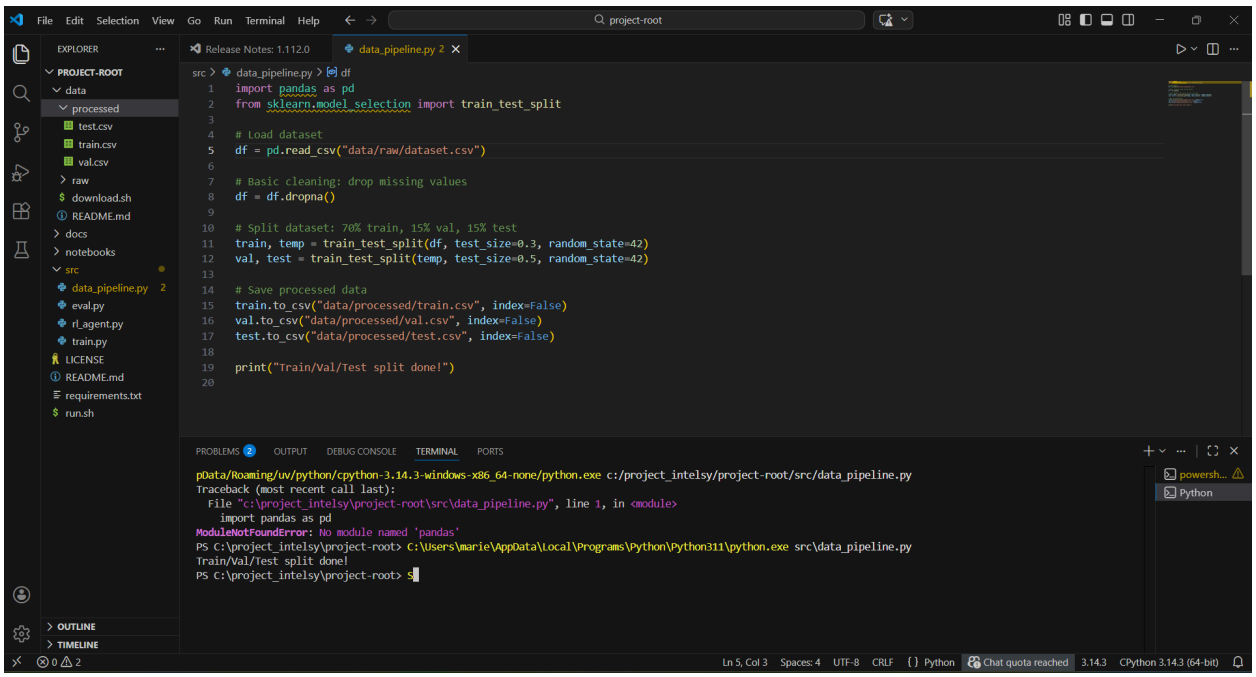
4. Open the EDA notebook:

jupyter notebook notebooks/01_eda.ipynb

5. Explore CNN experiments, NLP scaffold, and RL stub as needed.
   All Week 2 deliverables have been implemented, verified, and documented

CODE OUTPUT ***(screenshots)***

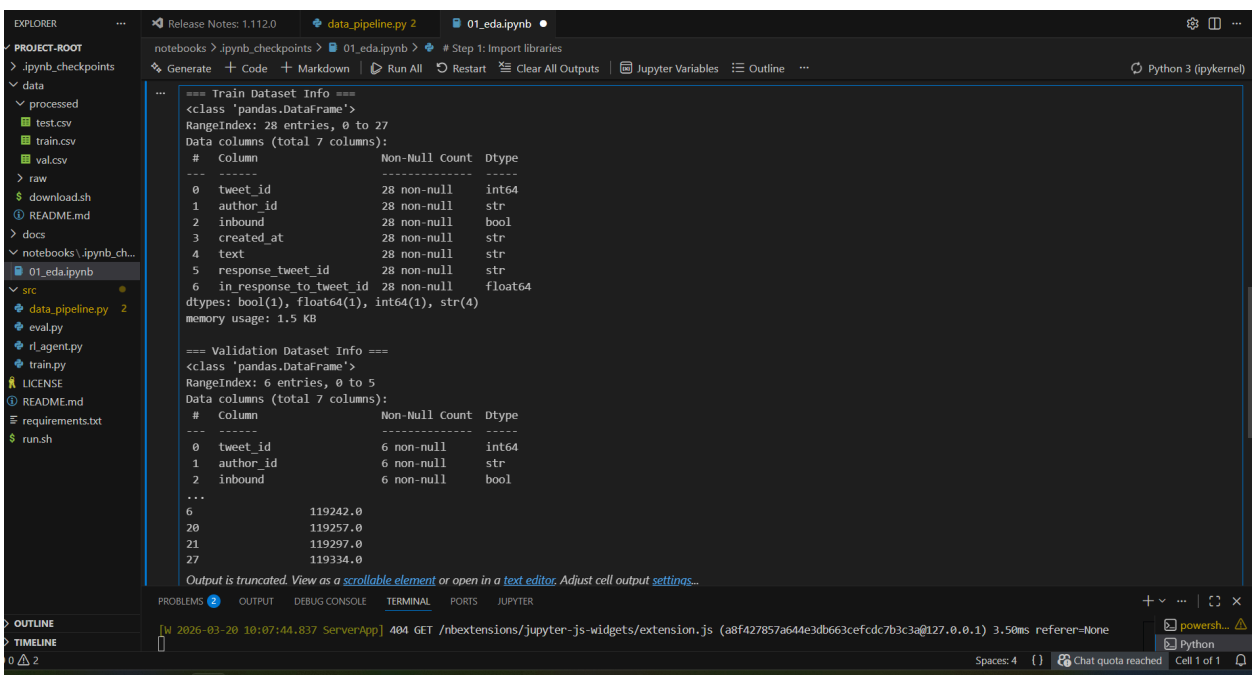
Train Validate and Test dataset

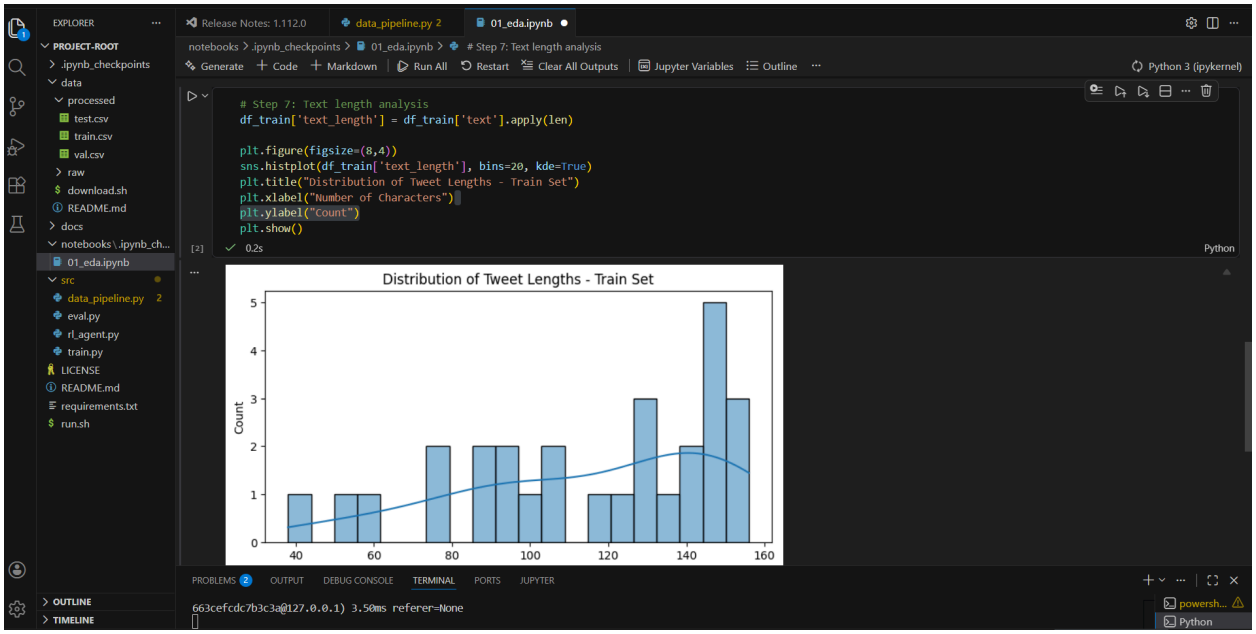


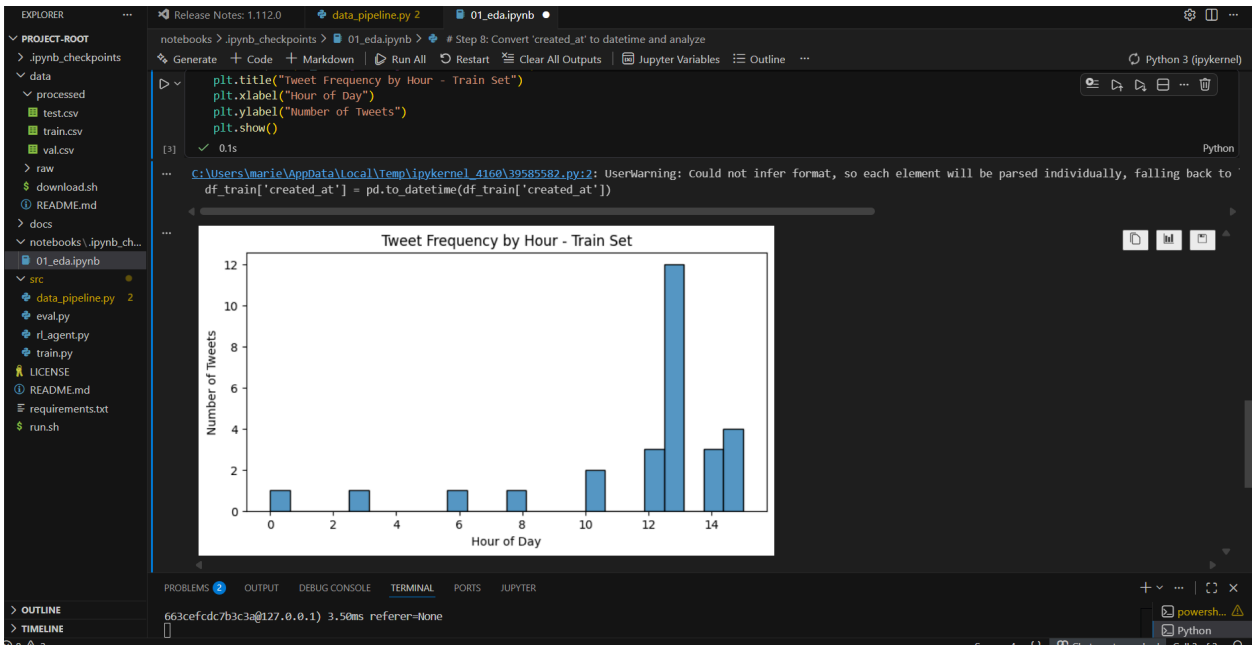
Train dataset in 01_notebook.ipynd EDA

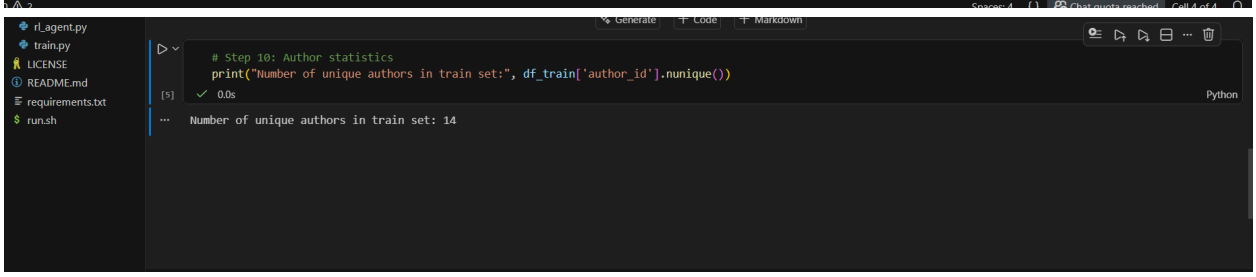
```
# Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load datasets
df_train = pd.read_csv("data/processed/train.csv")
df_val = pd.read_csv("data/processed/val.csv")
df_test = pd.read_csv("data/processed/test.csv")

# Step 3: General info
print("=== Train Dataset Info ===")
df_train.info()
```

```python
print("\n=== Validation Dataset Info ===")
df_val.info()
print("\n=== Test Dataset Info ===")
df_test.info()

# Step 4: Check for missing values
print("\n=== Missing Values (Train) ===")
print(df_train.isnull().sum())
print("\n=== Missing Values (Validation) ===")
print(df_val.isnull().sum())
print("\n=== Missing Values (Test) ===")
print(df_test.isnull().sum())

# Step 5: Sample few rows
print("\n=== Train Sample Messages ===")
print(df_train.sample(5))

# Step 6: Check class distribution (if you have a 'label' column)
if 'label' in df_train.columns:
    print("\n=== Class Distribution (Train) ===")
    print(df_train['label'].value_counts())

    # Optional: plot
    plt.figure(figsize=(6,4))
    sns.countplot(x='label', data=df_train)
    plt.title("Class Distribution - Train Set")
    plt.show()
```

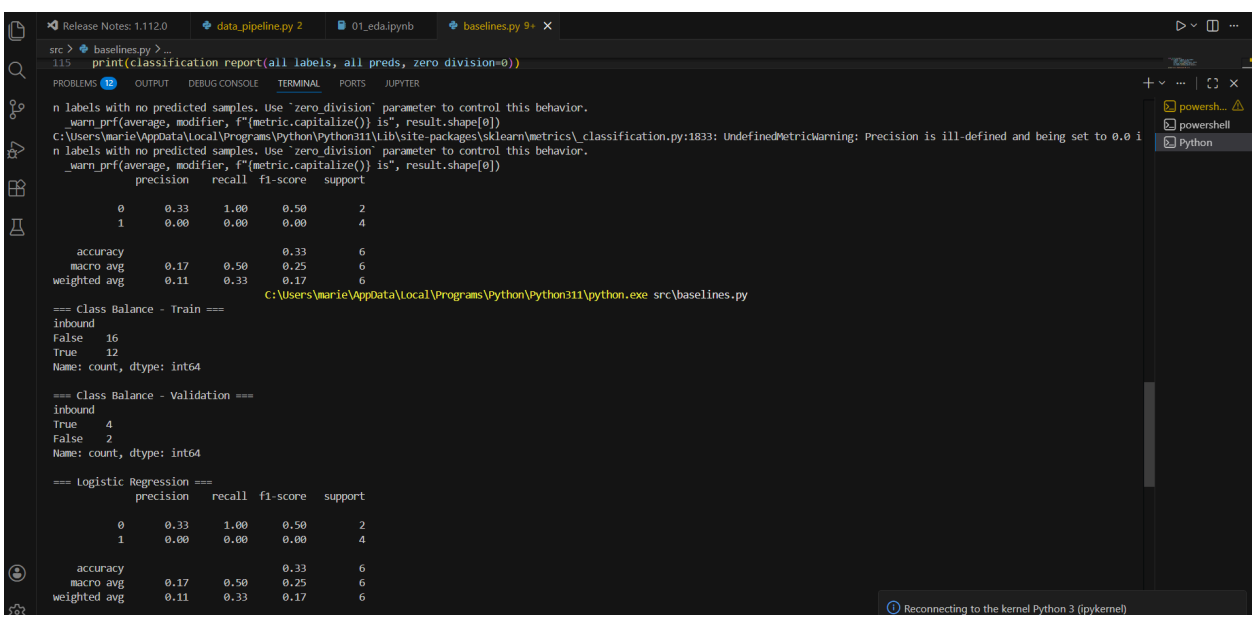


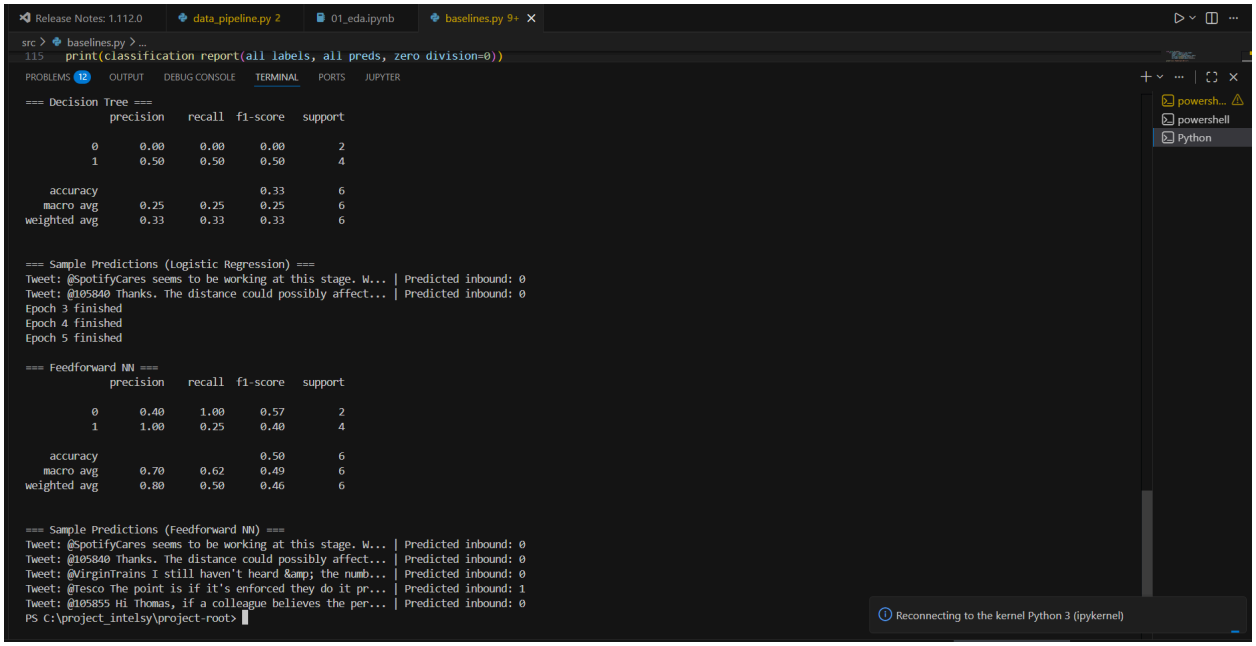

EXPLORER
PROJECT-ROOT
.ipynb_checkpoints
data
processed
test.csv
train.csv
val.csv
raw
download.sh
README.md
docs
notebooks\.ipynb_ch...
01_eda.ipynb
src
data_pipeline.py 2
eval.py
rl_agent.py
train.py
LICENSE
README.md
requirements.txt
run.sh
OUTLINE
TIMELINE
Release Notes: 1.112.0
data_pipeline.py 2
01_eda.ipynb
notebooks > .ipynb_checkpoints > 01_eda.ipynb > # Step 7: Text length analysis
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3 (ipykernel)
Step 7: Text length analysis
df_train['text_length'] = df_train['text'].apply(len)
plt.figure(figsize=(8,4))
sns.histplot(df_train['text_length'], bins=20, kde=True)
plt.title("Distribution of Tweet Lengths - Train Set")
plt.xlabel("Number of Characters")
plt.ylabel("Count")
plt.show()
[2]
0.2s
Python
Distribution of Tweet Lengths - Train Set
Count
0
1
2
3
4
5
40
60
80
100
120
140
160
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
JUPYTER
663cefcdc7b3c3a@127.0.0.1) 3.50ms referer=None
powersh...
Python


EXPLORER
PROJECT-ROOT
.ipynb_checkpoints
data
processed
test.csv
train.csv
val.csv
raw
download.sh
README.md
docs
notebooks\.ipynb_ch...
01_eda.ipynb
src
data_pipeline.py 2
eval.py
rl_agent.py
train.py
LICENSE
README.md
requirements.txt
run.sh
OUTLINE
TIMELINE
Release Notes: 1.112.0
data_pipeline.py 2
01_eda.ipynb
notebooks > .ipynb_checkpoints > 01_eda.ipynb > # Step 8: Convert 'created_at' to datetime and analyze
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3 (ipykernel)
plt.title("Tweet Frequency by Hour - Train Set")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Tweets")
plt.show()
[3]
0.1s
Python
C:\Users\marie\AppData\Local\Temp\ipykernel_4160\39585582.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to
df_train['created_at'] = pd.to_datetime(df_train['created_at'])
Tweet Frequency by Hour - Train Set
Number of Tweets
0
2
4
6
8
10
12
0
2
4
6
8
10
12
14
Hour of Day
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
JUPYTER
663cefcdc7b3c3a@127.0.0.1) 3.50ms referer=None
powersh...
Python

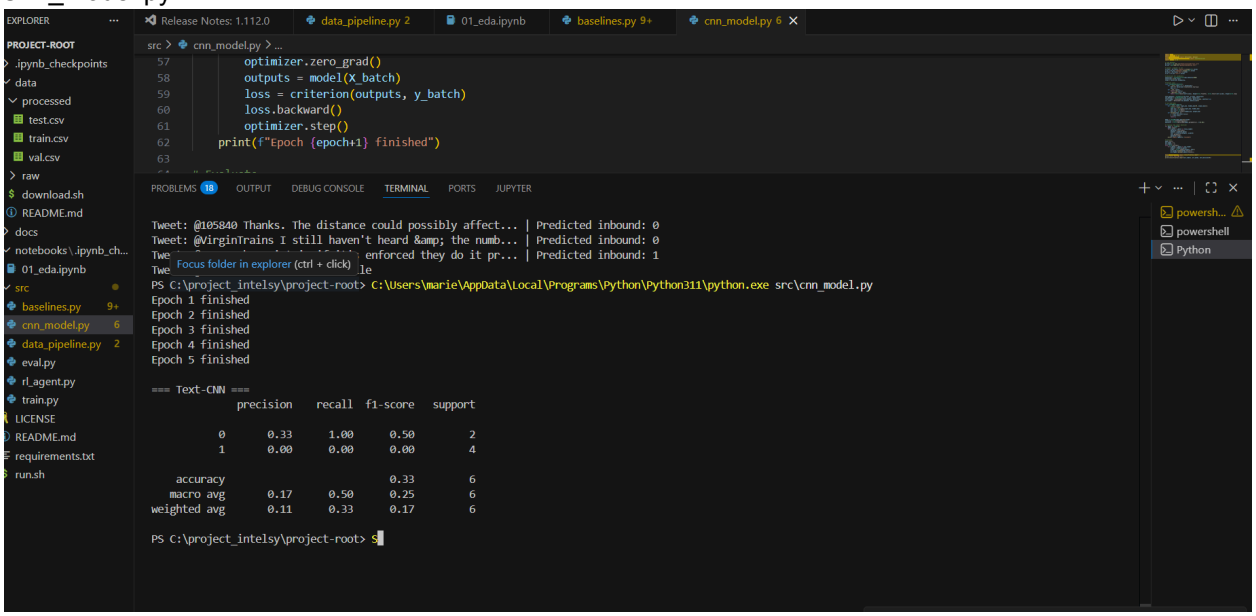


[Baselines.py]





Cnn_model.py



Nlp_scaffold.py

```python
sample_text = "Hello, I need help with my account"
predicted_intent = classifier.predict(sample_text)
print(f"Text: {sample_text}")
print(f"Predicted Intent: {predicted_intent}")
```

```
Tweet: @105840 Thanks. The distance could possibly affect... | Predicted inbound: 0
Tweet: @VirginTrains I still haven't heard &amp; the numb... | Predicted inbound: 0
Tweet: @Tesco The point is if it's enforced they do it pr... | Predicted inbound: 1
Tweet: @105855 Hi Thomas, if a colle
PS C:\project_intelsy\project-root> C:\Users\marie\AppData\Local\Programs\Python\Python311\python.exe src\cnn_model.py
Epoch 1 finished
Epoch 2 finished
Epoch 3 finished
Epoch 4 finished
Epoch 5 finished

=== Text-CNN ===
              precision    recall  f1-score   support

           0       0.33      1.00      0.50         2
           1       0.00      0.00      0.00         4

    accuracy                           0.33         6
   macro avg       0.17      0.50      0.25         6
weighted avg       0.11      0.33      0.17         6

PS C:\project_intelsy\project-root> C:\Users\marie\AppData\Local\Programs\Python\Python311\python.exe src\nlp_scaffold.py
Text: Hello, I need help with my account
Predicted Intent: info
PS C:\project_intelsy\project-root> S
```

Rl_stubpy

```python
for episode in range(3):
    action = random.choice(env.actions)
    next_state, reward, done = env.step(action)
    print(f"Episode {episode+1} | State: {state}, Action: {action}, Reward: {reward}")
    state = next_state
```

```
Tweet: @105840 Thanks. The distance could possibly affect... | Predicted inbound: 0
Tweet: @VirginTrains I still haven't heard &amp; the numb... | Predicted inbound: 0
Tweet: @Tesco The point is if it's enforced they do it pr... | Predicted inbound: 1
Tweet: @105855 Hi Thomas, if a colle
PS C:\project_intelsy\project-root> C:\Users\marie\AppData\Local\Programs\Python\Python311\python.exe src\cnn_model.py
Epoch 1 finished
Epoch 2 finished
Epoch 3 finished
Epoch 4 finished
Epoch 5 finished

=== Text-CNN ===
              precision    recall  f1-score   support

           0       0.33      1.00      0.50         2
           1       0.00      0.00      0.00         4

    accuracy                           0.33         6
   macro avg       0.17      0.50      0.25         6
weighted avg       0.11      0.33      0.17         6

PS C:\project_intelsy\project-root> C:\Users\marie\AppData\Local\Programs\Python\Python311\python.exe src\nlp_scaffold.py
Text: Hello, I need help with my account
Predicted Intent: info
PS C:\project_intelsy\project-root> C:\Users\marie\AppData\Local\Programs\Python\Python311\python.exe src\rl_stub.py
Episode 1 | State: message_2, Action: reply_bad, Reward: -1
Episode 2 | State: message_2, Action: reply_good, Reward: 1
Episode 3 | State: message_2, Action: reply_good, Reward: 1
PS C:\project_intelsy\project-root>
```